

99/00

67101

7

האוניברסיטה העברית בירושלים
המכון למדעי המחשב

בחינה במבוא למדעי המחשב
קורס מס' 67101

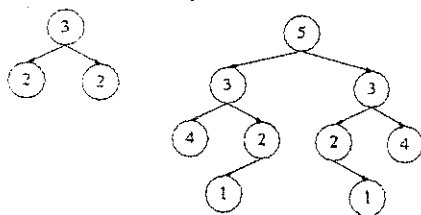
תאריך: 6.2.00
זמן: 2 ½ שעות

מועד א' תש"ס
המורה: פרופ' ג'ף רוזנשיין

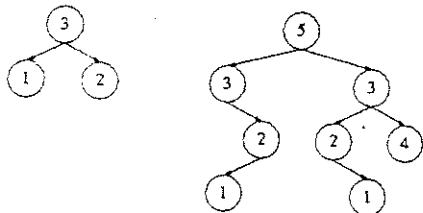
ענה על שלוש שאלות מתוך ארבע.

1. עליכם לממש פונקציה רקורסיבית שמקבלת עץ בינארי (לא בהכרח עץ חיפוש) ומחליטה האם העץ סימטרי או לא.

Example Of Symmetric Trees:



Example Of Asymmetric Trees:



העץ ממומש ע"י המחלקה הבאה:

```
public class BT {          // BT = Binary Tree
    public BT left;        // left son (null if doesn't exist)
    public BT right;       // right son (null if doesn't exist)
    public int data;       // data of this node
}
```

להלן ה-prototype של הפונקציה שעליכם לממש:

```
public class SymTree {
    public static boolean isSymmetric(BT bt) {
        // this is the method you must implement ...
        // it returns true if and only if bt is a symmetric tree.
        // You will probably want to use another method as well.
        // Do not use additional data structures.
    }
    // ...
}
```

99/00

6701

(8)

2. א. (28 נק') לרשותכם המחלקות הבאות:

```

public class IntNode {
    private int _data;
    private IntNode _next;

    public IntNode(int data) {
        _data = data;
        _next = null;
    }

    public void addNext(IntNode node) { _next = node; }
    public IntNode getNext() { return _next; }
    public int getData() { return _data; }
    public void removeNext() {
        IntNode tmp = getNext();
        if(tmp != null) {
            addNext(tmp.getNext());
            tmp._next = null;
        }
    }
}

```

```

public class IntList {
    private IntNode _head;

    public IntList() { _head = null; }
    public IntNode getHead() { return _head; }
    public void add(int data) {
        IntNode tmp;
        IntNode newNode = new IntNode(data);
        if(_head == null)
            _head = newNode;
        else {
            tmp = _head;
            while(tmp.getNext() != null)
                tmp = tmp.getNext();
            tmp.addNext(newNode);
        }
    }
}

```

```

public class TreeNode {
    private int _data;
    protected TreeNode _left, _right;

    public TreeNode(int data) {
        _data = data;
        _left = _right = null;
    }

    public void addRight(TreeNode node) { _right = node; }
    public void addLeft(TreeNode node) { _left = node; }
    public TreeNode getRight() { return _right; }
    public TreeNode getLeft() { return _left; }
    public int getData() { return _data; }
    public void setData(int newData) { _data = newData; }
}

```

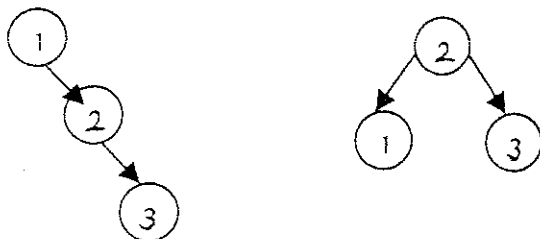
```

public class BinSearchTree {
    private TreeNode _root;

    public BinSearchTree() { _root = null; }
    public boolean add(int data) { ... }
    public boolean equals(BinSearchTree other) { }
}

```

נאמר ששני עצי חיפוש בינאריים שווים אם הם מכילים את אותם נתונים לדוגמא:



כתבו שיטה יעילה שכותרתה `public boolean equals(BinSearchTree other)` אשר תחזיר `true` אם העצים שווים אחרת תחזיר `false`.

ב. (5 נק') מה הסיבוכיות של השיטה שמימשתם?

3. נתונה התוכנית הבאה :

```
class A {
    public void f() {}
}

class B extends A {
    public void f() {System.out.println("hi");}
}

class C extends A {
    public void f() {System.out.println("bye");}
}

class Main {
    static public void f(A[] array) {
        array = new A[1];
        array[0] = new B();
    }

    static public void main(String[] args) {
        A[] array = new A[1];
        array[0] = new C();
        f(array);
        array[0].f();
    }
}
```

א. מה יהיה פלט התוכנית ומדוע ?

ב. בהנחה שמחליפים את הקוד של class A בקוד הבא :

```
abstract class A {
    abstract void f();
}
```

האם התוכנית תעבור קומפילציה ? אם לא, מדוע ? אם כן, מה יהיה הפלט ?

ג. מה יהיה הפלט במידה ונחליף בתוכנית המקורית את הפונקציה (השיטה) f() של מחלקה main בפונקציה הבאה :

```
static void f(A[] array) {
    array[0] = new B();
}
```

הסבירו מדוע.

4. נתון מערך של integers חיוביים בגודל $6 \times n$. המערך מייצג n מלבנים במישור שעלולים להסתיר זה את זה. כל קבוצה של 6 integers מייצגת שלוש נקודות המהוות קודקודים של מלבן : שמאלית עליונה, ימנית עליונה וימנית תחתונה. תחום הערכים של ה - integers אינו ידוע מראש.

המלבן הראשון עלול להסתיר חלק מכל מלבן אחר, המלבן השני עלול להסתיר חלק מהמלבנים $n \dots 3$ וכ'.

עליכם לכתוב את שיטה :

`public void rectPaint(int myArg)`

השיטה מקבלת מערך מהסוג שתואר, היא יוצרת מערך נוסף שיהיה גדול מספיק כדי להכיל את כל המלבנים, לאחר מכן, עליה למלא את אותו מערך חדש שיצרה במלבנים ואז להדפיס למסך את המערך החדש. ניתן לכתוב שיטות נוספות לביצוע המשימה. במידה ואורך המערך `myArg` אינו כפולה שלמה של 6, יש להתעלם ממספר נתונים בסוף המערך. מספר זה יהיה שארית החלוקה של אורך המערך ב - 6.

לדוגמה :

אם השיטה קיבלה את המערך הבא :

2	1	2	5	4	5	3	4	3	6	6	6	1	2	1	3	2	3
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

אורך המערך הוא 18 ולכן הוא מייצג שלושה מלבנים. הפינה השמאלית עליונה של המלבן הראשון היא (2,1) והפינה הימנית תחתונה שלו היא (4,5). הפינה השמאלית עליונה של המלבן השני היא (3,4), הפינה הימנית עליונה שלו היא (3,6) והפינה הימנית תחתונה היא (2,3). ניתן להניח כי כל המספרים במערך זה אכן מייצגים מלבנים במישור (מבחינה זו הקלט תקין).

עבור הדוגמה שהבאנו, השיטה תיצור מערך של 7×7 שיחזיק את שלושת המלבנים ותדפיס את הפלט הבא (השורות ממוספרות מלמעלה למטה) :

0	0	0	0	0	0	0
0	0	3	3	0	0	0
0	1	1	1	1	1	0
0	1	1	1	1	1	2
0	1	1	1	1	1	2
0	0	0	0	2	2	2
0	0	0	0	2	2	2

בהצלחה!